掌握回测引擎Backtrader

>> 今天的学习目标

掌握回测引擎Backtrader

- 什么是 Backtrader?
- Backtrader 的核心组件
- Cerebro 引擎的标准工作流 (SOP)
- CASE：趋势跟踪策略（双均线）
- Backtrader 策略类的三大核心模块解析
- 回测评估指标

总收益率、年化收益率、最大回撤、夏普比率、胜率、盈亏比

- CASE：MACD 策略（趋势跟踪之王）
- CASE：RSI 策略（超买超卖的波段利器）
- CASE：布林带策略（捕捉波动率的橡皮筋）
- CASE：乖离率策略（均值回归的极限拉扯）
- CASE：动量策略（追涨杀跌的强者恒强）
- 线上系统的动态策略插件机制
- CASE：MACD 底背离策略开发

什么是 Backtrader?

Thinking：什么是Backtrader?

Backtrader 是一个基于 Python 的开源量化交易策略回测框架，广泛应用于算法交易策略的开发。

它充当系统的时间机器——让开发者在历史数据上模拟交易策略的逐日（或逐分钟）执行

=> 评估在过去市场环境中的表现

=> 用数据验证策略逻辑是否具备有效性

什么是 Backtrader?

Backtrader的主要特点：

- 事件驱动架构：模拟真实市场逐根 K 线（bar-by-bar）的处理方式，支持复杂的挂单逻辑，如止损、止盈、条件触发单等。
- 高度灵活性：支持股票、期货、外汇、加密货币等多种资产类别；支持自定义底层指标、分析器和数据观察者。
- 内置技术指标库：原生集成移动平均（SMA/EMA）、MACD、RSI、布林带等主流技术分析指标，大幅降低代码编写量。
- 多数据源支持：可同时接入多个时间周期（如日线与周线结合的多周期策略），或多个交易标的（如配对交易或资产配置策略）。
- 详尽的性能分析：内置多种 Analyzer（分析器），自动计算并生成夏普比率、最大回撤、胜率、盈亏比等核心统计报告。

Backtrader 的核心组件

Thinking：在Backtrader中 有哪些核心组件?

- Cerebro（回测引擎的大脑）：负责统筹协调数据流、策略执行、经纪商撮合和分析器计算的完整生命周期。 => 调度系统
- Data Feed（数据源）：负责将外部数据（如 CSV、MySQL 数据库或 Pandas DataFrame）标准化并持续喂入引擎。=> 行情数据源
- Strategy（策略类）：量化研究员编写核心交易逻辑的地方（定义开仓、平仓、仓位管理及指标计算规则）=> 交易大脑
- Broker（模拟经纪商）：处理订单的路由与撮合，管理账户的虚拟资金、计算佣金（Commission）及模拟滑点（Slippage）=> 券商通道
- Analyzer（分析器）：挂载在 Cerebro 上，在回测结束后对策略的资金曲线和交易日志进行多维度绩效计算 => 绩效统计

Cerebro 引擎的标准工作流 (SOP)

在一个标准的量化工程中，启动回测通常遵循严格的 4 步初始化流程：

Step1，引擎初始化

实例化 Cerebro 控制中心。

```
import backtrader as bt


# 1. 实例化 Cerebro 大脑
cerebro = bt.Cerebro()
```

data_loader.py

Cerebro 引擎的标准工作流 (SOP)

Step2，数据接入

将清洗好的结构化数据转换成 Backtrader 兼容的数据流格式。

```
# 假设 df 是从 MySQL 读取并清洗后的 Pandas DataFrame
data = bt.feeds.PandasData(dataname=df)


# 2. 将数据流挂载到 Cerebro
cerebro.adddata(data)
```

data_loader.py

Cerebro 引擎的标准工作流 (SOP)

Step3：策略挂载

将开发好的策略类（如双均线策略）注册到引擎中。

Cerebro 会在运行期间自动调用该策略的 next() 方法逐行处理 K 线。

```
# 3. 添加交易策略
cerebro.addstrategy(DoubleMAStrategy)
```

data_loader.py

Cerebro 引擎的标准工作流 (SOP)

Step4：环境与规则配置

这是量化回测中最容易被忽视，会影响实盘胜率的环节。 需要配置真实的交易摩擦成本。

```
# 4. 配置虚拟经纪商账户
cerebro.broker.setcash(1000000.0)                # 设定初始启动资金
cerebro.broker.setcommission(commission=0.0002)      # 设定双边万分之二的手续费
cerebro.addsizer(bt.sizers.PercentSizer, percents=95) # 设定固定仓位管理机制（每次动用 95% 资金）
```

很多量化新手在回测时容易陷入真空环境测试的误区（即不设置手续费和滑点）。

对于某些高频交易或动量策略，如果一年可能产生数百次交易，累积的摩擦成本（印花税、佣金）很有可能将一个在账面上盈利 30% 的策略直接打成亏损。

=> 设定 setcommission 确保回测结果的有效性

data_loader.py

CASE：趋势跟踪策略（双均线）

TO DO：搭建双均线策略

在 Backtrader 中，所有的策略都必须继承自基类 bt.Strategy。

为了快速上手，我们以经典的双均线策略（Double Moving Average）为例，

核心思想：利用不同周期的移动平均线（MA）的相对位置，判断市场趋势的启动与终结。

买入条件（多头开仓）：当代表短期趋势的快线（如 10 日均线）由下向上穿过代表长期趋势的慢线（如 30 日均线）时，形成金叉，视为上涨趋势确立，触发全仓买入。

卖出条件（多头平仓）：当快线由上向下穿过慢线时，形成死叉，视为上涨趋势结束或下跌趋势开启，触发清仓止盈/止损。

CASE：趋势跟踪策略（双均线）

```
import backtrader as bt

class DoubleMAStrategy(bt.Strategy):
    """双均线金叉/死叉策略"""

    # 1. 参数定义区：将策略参数暴露出来，便于后续做参数寻优
    params = (('fast', 10), ('slow', 30))

    def __init__(self):
        # 2. 初始化区：声明指标与计算图
        self.ma_fast = bt.indicators.SMA(self.data.close, period=self.p.fast)
        self.ma_slow = bt.indicators.SMA(self.data.close, period=self.p.slow)

        # CrossOver 指标：快线上穿慢线返回 1，下穿返回 -1，无交叉返回 0
        self.crossover = bt.indicators.CrossOver(self.ma_fast, self.ma_slow)
```

```
    def next(self):
        # 3. 步进逻辑区：按时间序列逐根 K 线执行的交易逻辑
        if not self.position:  # 状态机：如果当前账户处于空仓状态
            if self.crossover > 0:  # 捕捉到金叉信号
                self.buy()        # 触发买入指令（由引擎按设定的百分比测算股数）
        elif self.crossover < 0:   # 状态机：如果当前持仓，且捕捉到死叉信号
            self.close()          # 触发平仓指令
```

bt.indicators.SMA　　调用 backtrader 内置的简单移动平均线指标

self.data.close　　使用当前数据源的收盘价作为计算输入

period=self.p.fast　　设置计算周期，值来自策略参数 self.p.fast（通常定义为 5、10、20 等短期周期）

self.ma_fast　　将指标实例保存为策略实例的属性，方便在 next() 中引用

1-双均线策略.py

Backtrader 策略类的三大核心模块解析

模块一：params（参数配置）

使用元组的形式定义策略的全局变量（如周期快慢线参数 10 和 30）。

将硬编码（Hard-code）参数剥离。

=> 在后续的应用中，Cerebro 可以通过网格搜索（Grid Search）等方式，自动遍历这些 params

=> 寻找使策略夏普比率最高的最佳参数组合。

代码内部通过 self.p.fast 或 self.params.fast 进行调用。

1-双均线策略.py

Backtrader 策略类的三大核心模块解析

模块二：__init__()（声明与预计算逻辑）

__init__ 是在回测正式启动前、对象实例化时执行的。

这里不执行任何交易动作

=> 作用是向框架声明需要用到哪些技术指标（如 bt.indicators.SMA 和 bt.indicators.CrossOver）。

- 性能优化机制：

Backtrader 底层会自动根据声明的指标，构建计算图并进行向量化预计算。

不要在下一节的 next() 循环里去实例化指标，那会导致极大的性能浪费。

1-双均线策略.py

Backtrader 策略类的三大核心模块解析

模块三：next()（事件驱动的业务逻辑）

这是策略的心脏。Cerebro 引擎在推进历史数据的过程中，每走完一根 K 线（如每个交易日收盘），就会调用一次 next() 方法。

- 仓位状态机（self.position）：

在发送交易指令前，系统会先判断当前的持仓状态（空仓还是多头）。

=> 防止在已有仓位的情况下重复发出买入指令。

- 信号执行：

依赖 __init__ 中计算好的 self.crossover 信号数值（1 表示金叉，-1 表示死叉）。

结合当前的仓位状态，使用 self.buy() 和 self.close() 函数进行订单的路由分发。

1-双均线策略.py

CASE：趋势跟踪策略（双均线）

股票: 600519.SH

数据: 2025-01-02 ~ 2025-12-31 共243个交易日

==========

初始资金: 1,000,000.00

手续费: 2.0 (万分之)

仓位比例: 95%

最终资金: 910,551.59

总收益率: -8.94%

最大回撤: 10.36%

夏普比率: 0.0000

交易次数: 4

胜率: 25.00%

==========

1-双均线策略.py

绩效报告核心指标解析

当 cerebro.run() 执行完毕后，系统会吐出大量的交易流水和最终的账户资金情况。

在AI量化中，我们需要对量化策略进行评估

=> 通过向 Cerebro 添加各种 bt.analyzers（分析器）

=> 自动化提取多维度的绩效数据。

Thinking：量化交易策略的衡量都有哪些关键指标?

- 收益评估：总收益率 vs 年化收益率
- 风险控制：最大回撤
- 性价比量化：夏普比率
- 交易质量：胜率 vs 盈亏比

收益评估：总收益率 vs 年化收益率

- 收益评估：总收益率 vs 年化收益率

很多人在测试策略时，容易被绝对的总收益率迷惑。

总收益率计算：(最终资金 - 初始资金) / 初始资金。

如果一个策略跑了 10 年，从 100 万变成 200 万，总收益率是 100%。

年化收益率 (CAGR)：这是将不同时间跨度的策略拉到同一条起跑线上比拼的关键指标。

$$\text{CAGR} = (1 + \text{Total Return})^{\frac{1}{\text{Years}}} - 1$$

上述 10 年赚 100% 的策略，年化收益率其实只有约 7.18%，

=> 甚至跑不赢某些优质的底层资产盲目持有。

在实战中，我们要用年化收益率来衡量策略的盈利效率。

收益评估：总收益率 vs 年化收益率

Thinking：初始价值：¥100,000，5 年后价值：¥180,000

请问：总收益率=? 年化收益 = ?

总收益率 = (180000-100000) / 100000 = 0.8

$$\text{CAGR} = (1+0.80)^{\frac{1}{5}} - 1 = 1.80^{0.2} - 1 \approx 1.1249 - 1 = \mathbf{12.49\%}$$

Thinking：为什么要用年化收益率?

- 几何平均：反映资金的实际复利增长路径，不同于算术平均（简单平均每年收益）
- 平滑波动：忽略中间波动，只关注起点和终点的价值变化
- 跨资产比较：便于比较不同持有期的投资表现（如对比3年期基金 vs 5年期债券）

风险控制：最大回撤

- 风险控制：最大回撤

定义：资金曲线从历史最高点滑落到最低点的最大跌幅百分比。

在我们的底层代码中，这是通过 bt.analyzers.DrawDown 来计算的。

核心逻辑：这是实盘中最考验人性的指标。

Thinking：如果一个策略的年化收益有 30%，但最大回撤高达 40%，你能拿住么？

在遇到极端行情时，你的 100 万账户会缩水到 60 万。很多投资者还没熬过最大回撤期，就已经平仓了。

收益衡量的是策略的上限，而回撤衡量的是策略的下限。

风险控制：卡玛比率

- 卡玛比率 (Calmar Ratio)：

为了综合评估，我们将年化收益除以最大回撤，得出卡玛比率。

比值越高，说明策略在承受同等回撤下获取收益的能力越强。

卡玛比率范围	评级	含义
> 3.0	⭐⭐⭐⭐⭐	卓越，风险收益比极佳（优秀CTA水平）
2.0 - 3.0	⭐⭐⭐⭐	优秀，风险控制好，收益稳定
1.0 - 2.0	⭐⭐⭐	良好，大多数稳健策略的区间
0.5 - 1.0	⭐⭐	一般，收益与风险勉强匹配
< 0.5	⭐	较差，风险过高或收益不足

优秀的 CTA 标准：卡玛比率 > 2.0（顶级可达 3-5）

年化收益 15-30%，但最大回撤控制在 10% 以内

性价比量化：夏普比率

- 性价比量化：夏普比率

定义：每承担 1 份额外风险（波动率），能换来几份超额收益。

代码中由 bt.analyzers.SharpeRatio 引擎计算（通常无风险利率设定为 0.02）。

评估标准：

- < 1.0：策略承担的风险没有得到相应的收益补偿，通常不建议实盘。
- > 1.0：合格的策略。
- > 2.0：非常优秀的策略。

夏普比例是衡量风险调整后收益的行业标杆

简单来说，夏普比率不看你最后赚了多少，而是看你赚得过程有多平稳。

交易质量：胜率 vs 盈亏比

- 交易质量：胜率 vs 盈亏比

系统通过 bt.analyzers.TradeAnalyzer 提取所有的历史订单进行统计。

胜率 (Win Rate)：盈利交易次数占总交易次数的比例。

胜率 = 胜场数 / 总交易次数。

盈亏比 (P/L Ratio)：平均每次盈利的金额除以平均每次亏损的金额的绝对值

盈亏比 = abs(avg_win / avg_loss))。

胜率 和 盈亏比这两个指标直接决定了策略的内功底子

一键化报告输出

为了避免大家在日常回测中写大量重复的打印代码，我在工具包 data_loader.py 中封装了统一的方法。

```
run_and_report(DoubleMAStrategy, '600519.SH', '2024-01-01', '2025-12-31', label='双均线策略', plot=True)
```

调用run_and_report：系统就会自动调用上述所有的 Analyzer，不仅能在控制台打印出整齐的指标对齐报告，还能将 K 线买卖点、净值曲线对比和回撤阴影图自动保存到本地文件夹。

经典指标矩阵

掌握了 Backtrader 的底层运行框架和双均线策略后，你会发现：量化回测的本质，就是像搭积木一样替换不同的技术指标。

⇒核心逻辑几乎一模一样，只需替换 bt.indicators 中的算子即可。

Thinking：都有哪些常用的策略？

- 趋势跟踪：双均线、MACD策略、三重指数均线
- 超买超卖：RSI策略、DKJ策略、顺势指标
- 突破系统：海龟交易法则、唐奇安通道、放量突破
- 波动率：布林带策略、ATR通道突破
- 均值回归：乖离率策略

CASE： MACD 策略（趋势跟踪之王）

TO DO：搭建MACD交易策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

核心参数：短周期 12，长周期 26，信号线（DEA）9。

交易规则：

- 买入条件：当快线（DIF）由下向上穿过慢线（DEA），形成金叉，代表多头动能增强，全仓买入。
- 卖出条件：当快线（DIF）由上向下穿过慢线（DEA），形成死叉，代表空头动能增强，清仓卖出。

MACD（指数平滑异同移动平均线）是实战中最常用的趋势跟踪指标之一。它通过计算两根不同周期的指数移动平均线（EMA）的差值，来判断市场趋势的动能。

CASE： MACD 策略（趋势跟踪之王）

```
import backtrader as bt
from data_loader import run_and_report

class MACDStrategy(bt.Strategy):
    # 1. 定义标准参数：12, 26, 9
    params = (('short', 12), ('long', 26), ('signal', 9))
    def __init__(self):
        # 2. 调用内置的 MACD 指标
        self.macd = bt.indicators.MACD(
            self.data.close,
            period_me1=self.p.short,
            period_me2=self.p.long,
            period_signal=self.p.signal)

        # 3. 提取 MACD 线（快线）和 Signal 线（慢线）进行交叉判断
        # macd.macd 代表 DIF 线，macd.signal 代表 DEA 线
        self.crossover = bt.indicators.CrossOver(self.macd.macd, self.macd.signal)
```

```
def next(self):
    # 4. 仓位状态机与买卖逻辑
    if not self.position:
        if self.crossover > 0:  # 金叉触发买入
            self.buy()
    elif self.crossover < 0:    # 死叉触发卖出
        self.close()
```

在 Backtrader 中，我们不需要手动去计算复杂的 EMA 差值，系统已经内置了 MACD 指标。

2-MACD策略.py

CASE： MACD 策略（趋势跟踪之王）

MACD策略 | 600519.SH | 2025-01-02 ~ 2025-12-31 | 243个交易日

总收益: -4.56% | 年化: -4.73% | 最大回撤: 9.14% | 夏普: 0.00 | 卡玛: -0.52

交易: 6次 | 胜率: 16.7% | 盈亏比: 1.79 | 利润因子: 0.45 | 最大连亏: 2次

CASE： RSI 策略（超买超卖的波段利器）

TO DO：搭建RSI交易策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

核心参数：RSI 观察周期 14，超卖线 30，超买线 70。

交易规则：

- 买入条件：当 RSI 数值低于 30 时（超卖区域）=> 短期内被过度抛售，随时可能触底反弹，触发买入。
- 卖出条件：当 RSI 数值高于 70 时（超买区域）=> 短期内市场情绪过热，随时可能面临回调，触发卖出平仓。

如果我们说均线和 MACD 是用来顺势而为，那么 RSI（相对强弱指标）就是用来寻找拐点。
它属于震荡指标，核心思想是：涨得太多总要跌，跌得太多总要涨。

CASE： RSI 策略（超买超卖的波段利器）

```
import backtrader as bt
from data_loader import run_and_report

class RSIStrategy(bt.Strategy):
    # 1. 定义标准参数：周期 14，超卖阈值 30，超买阈值 70
    params = (('period', 14), ('oversold', 30), ('overbought', 70))

    def __init__(self):
        # 2. 调用内置的 RSI 指标
        self.rsi = bt.indicators.RSI(self.data.close, period=self.p.period)
```

```
def next(self):
    # 3. 仓位状态机与买卖逻辑（基于绝对数值的阈值判断）
    if not self.position:
        # 如果空仓，且 RSI 小于超卖线，买入
        if self.rsi < self.p.oversold:
            self.buy()
    # 如果持仓，且 RSI 大于超买线，卖出
    elif self.rsi > self.p.overbought:
        self.close()
```

与 MACD 使用两条线进行交叉计算不同，RSI 是直接将指标当前的绝对数值与我们设定的静态阈值进行比对。

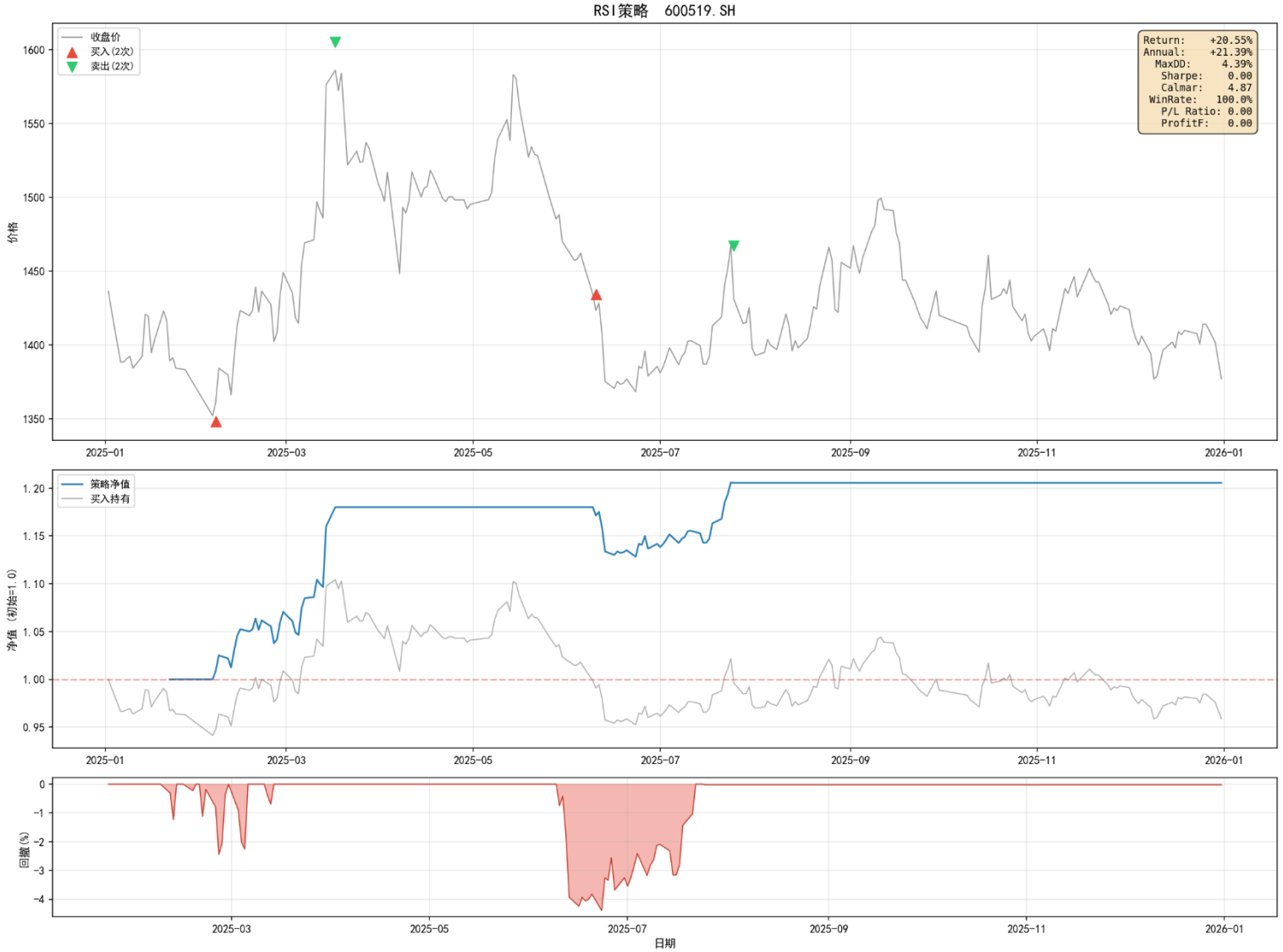
3-RSI策略.py

CASE： RSI 策略（超买超卖的波段利器）

RSI策略 | 600519.SH | 2025-01-02 ~ 2025-12-31 | 243个交易日

总收益: +20.55% | 年化: +21.39% | 最大回撤: 4.39% | 夏普: 0.00 | 卡玛: 4.87

交易: 2次 | 胜率: 100.0% | 盈亏比: 0.00 | 利润因子: 0.00 | 最大连亏: 0次



CASE：布林带策略（捕捉波动率的橡皮筋）

TO DO：搭建BOLL交易策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

核心参数：计算均线的周期为 20，上下轨的距离为 2.0 倍标准差。

交易规则：

- 买入条件：当当前收盘价跌破或触及下轨（均线 - 2倍标准差）时，认为价格被过度低估，触发买入。
- 卖出条件：当当前收盘价突破或触及上轨（均线 + 2倍标准差）时，认为价格被过度高估，触发卖出平仓。

如果说均线是市场的一根硬竹竿，那么布林带（Bollinger Bands）就是包裹着价格的一根弹性橡皮筋。它通过统计学中的标准差来动态计算出一条上轨和一条下轨，当价格暴涨暴跌触及轨线时，就像橡皮筋被拉到了极限，往往会产生向均值回归的拉力。

CASE：布林带策略（捕捉波动率的橡皮筋）

```
import backtrader as bt
from data_loader import run_and_report

class BollingerBandStrategy(bt.Strategy):
    # 1. 定义标准参数：中轨周期 20，标准差倍数 2.0
    params = (('period', 20), ('devfactor', 2.0))

    def __init__(self):
        # 2. 调用内置的布林带指标
        self.boll = bt.indicators.BollingerBands(
            self.data.close, period=self.p.period, devfactor=self.p.devfactor)
```

```
def next(self):
    # 3. 仓位状态机与买卖逻辑
    if not self.position:
        # 判断：当前收盘价 < 布林带下轨
        if self.data.close[0] < self.boll.bot[0]:
            self.buy()
    # 判断：当前收盘价 > 布林带上轨
    elif self.data.close[0] > self.boll.top[0]:
        self.close()
```

self.data.close[0] 代表当前这根 K 线（也就是今天）的收盘价。

self.boll.bot[0] 代表今天计算出的下轨数值

self.boll.top[0] 代表今天计算出的上轨数值

4-布林带策略.py

CASE：布林带策略（捕捉波动率的橡皮筋）

布林带策略 | 600519.SH | 2025-01-02 ~ 2025-12-31 | 243个交易日

总收益: +7.11% | 年化: +7.39% | 最大回撤: 2.49% | 夏普: 0.00 | 卡玛: 2.97

交易: 3次 | 胜率: 66.7% | 盈亏比: 0.00 | 利润因子: 0.00 | 最大连亏: 0次

CASE：乖离率策略（均值回归的极限拉扯）

TO DO：搭建乖离率交易策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

核心参数：基准均线周期 20，买入阈值（超跌）-6%，卖出阈值（超涨）6%。

交易规则：

- 买入条件：当乖离率 < -6% 时，代表价格短期内暴跌，过度偏离均线（超跌），随时可能报复性反弹
- 卖出条件：当乖离率 > 6% 时，代表价格短期内暴涨，严重透支未来空间（超涨），随时可能回调

在金融市场中，价格就像一只被主人（均线）牵着的小狗。虽然小狗会跑前跑后，但无论跑多远，最终都会被绳子拉回主人身边。这就是均值回归（Mean Reversion）的核心思想。
乖离率（BIAS）就是用来衡量这只小狗偏离主人有多远的指标。

CASE：乖离率策略（均值回归的极限拉扯）

乖离率（BIAS，也叫偏离率）是衡量股价与移动平均线偏离程度的动量指标。

$$\text{乖离率} = \frac{\text{当日收盘价} - N\text{日移动平均价}}{N\text{日移动平均价}} \times 100\%$$

- 正值：价格在均线上方，偏离越远代表短期涨幅越大（超买）
- 负值：价格在均线下方，偏离越远代表短期跌幅越大（超卖）
- 零值：价格正好落在均线上

```
# Backtrader 内置了 Bias 指标
def __init__(self):
    self.sma = bt.indicators.SMA(self.data.close, period=self.p.period)
    # 使用内置 Bias 指标
    self.bias = bt.indicators.Bias(self.data.close, period=self.p.period)
```

```
# 也可以手动计算，更直观
def __init__(self):
    self.sma = bt.indicators.SMA(self.data.close, period=self.p.period)
    # 手工计算：(收盘价 - 均线) / 均线 * 100
    self.bias = ((self.data.close - self.sma) / self.sma) * 100
```

CASE：乖离率策略（均值回归的极限拉扯）

```
import backtrader as bt
from data_loader import run_and_report

class BIASStrategy(bt.Strategy):
    # 1. 定义标准参数：20日均线，买入阈值 -6.0，卖出阈值 6.0
    params = (('period', 20), ('buy_threshold', -6.0), ('sell_threshold', 6.0))

    def __init__(self):
        # 2. 预计算基准：20日简单移动平均线 (SMA)
        self.sma = bt.indicators.SMA(self.data.close, period=self.p.period)
```

```
def next(self):
    # 3. 自定义动态计算：在 K 线循环中实时计算当天的乖离率
    bias = (self.data.close[0] - self.sma[0]) / self.sma[0] * 100

    # 4. 仓位状态机与买卖逻辑
    if not self.position:
        if bias < self.p.buy_threshold:  # 跌破 -6% 抄底
            self.buy()
    elif bias > self.p.sell_threshold:  # 涨破 6% 逃顶
        self.close()
```

在 __init__ 中，我们只声明了基础的 SMA 均线。

而在 next() 的逐日循环中，我们利用 self.data.close[0]（今天收盘价）和 self.sma[0]（今天均线价），直接通过 Python 的基础加减乘除算出了 bias 变量。

=>在 Backtrader 中，只要你有底层的数据（收盘价、开盘价、成交量等）和算子（均线、标准差等），你可以在 next() 函数里写出任何你想要的数学逻辑，甚至接入你自己训练的外部 AI 模型给出的得分。

5-乖离率策略.py

CASE：乖离率策略（均值回归的极限拉扯）

乖离率策略 | 600519.SH | 2025-01-02 ~ 2025-12-31 | 243个交易日

总收益: +1.89% | 年化: +1.96% | 最大回撤: 7.67% | 夏普: 0.00 | 卡玛: 0.26

CASE：动量策略（追涨杀跌的强者恒强）

TO DO：搭建动量交易策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

核心参数：观察周期 20 日，涨跌幅阈值 5%。

交易规则：

- 买入条件：当过去 20 天的累计涨幅突破 5% 时，说明资金正在强势介入，向上动能充足，顺势买入（追涨）。
- 卖出条件：当过去 20 天的累计跌幅超过 5% 时，说明趋势已经走坏，动能向下，果断卖出平仓（杀跌）。

在量化投资中，有两派截然相反的哲学。均值回归（乖离率）信奉的是物极必反，跌多了总会涨。
而动量策略（Momentum）信奉是：强者恒强，弱者恒弱 => 俗称追涨杀跌

CASE：动量策略（追涨杀跌的强者恒强）

```
import backtrader as bt
from data_loader import run_and_report

class MomentumStrategy(bt.Strategy):
    # 1. 定义标准参数：观察周期 20 天，触发阈值 5.0 (%)
    params = (('period', 20), ('threshold', 5.0))

    def __init__(self):
        # 2. 调用内置动量指标：计算 N 日变动率 (Rate of Change)
        self.roc = bt.indicators.ROC100(self.data.close, period=self.p.period)
```

```
def next(self):
    # 3. 仓位状态机与买卖逻辑
    if not self.position:
        # 判断：如果 20 日涨幅 > 5%，动能向上，追涨买入
        if self.roc[0] > self.p.threshold:
            self.buy()
    # 判断：如果 20 日跌幅 > 5%（即小于 -5%），动能向下，杀跌平仓
    elif self.roc[0] < -self.p.threshold:
        self.close()
```

动量策略的买入条件是 > 阈值 向上突破买入

而均值回归（乖离率）的买入条件是 < 负阈值 向下暴跌后抄底买入。

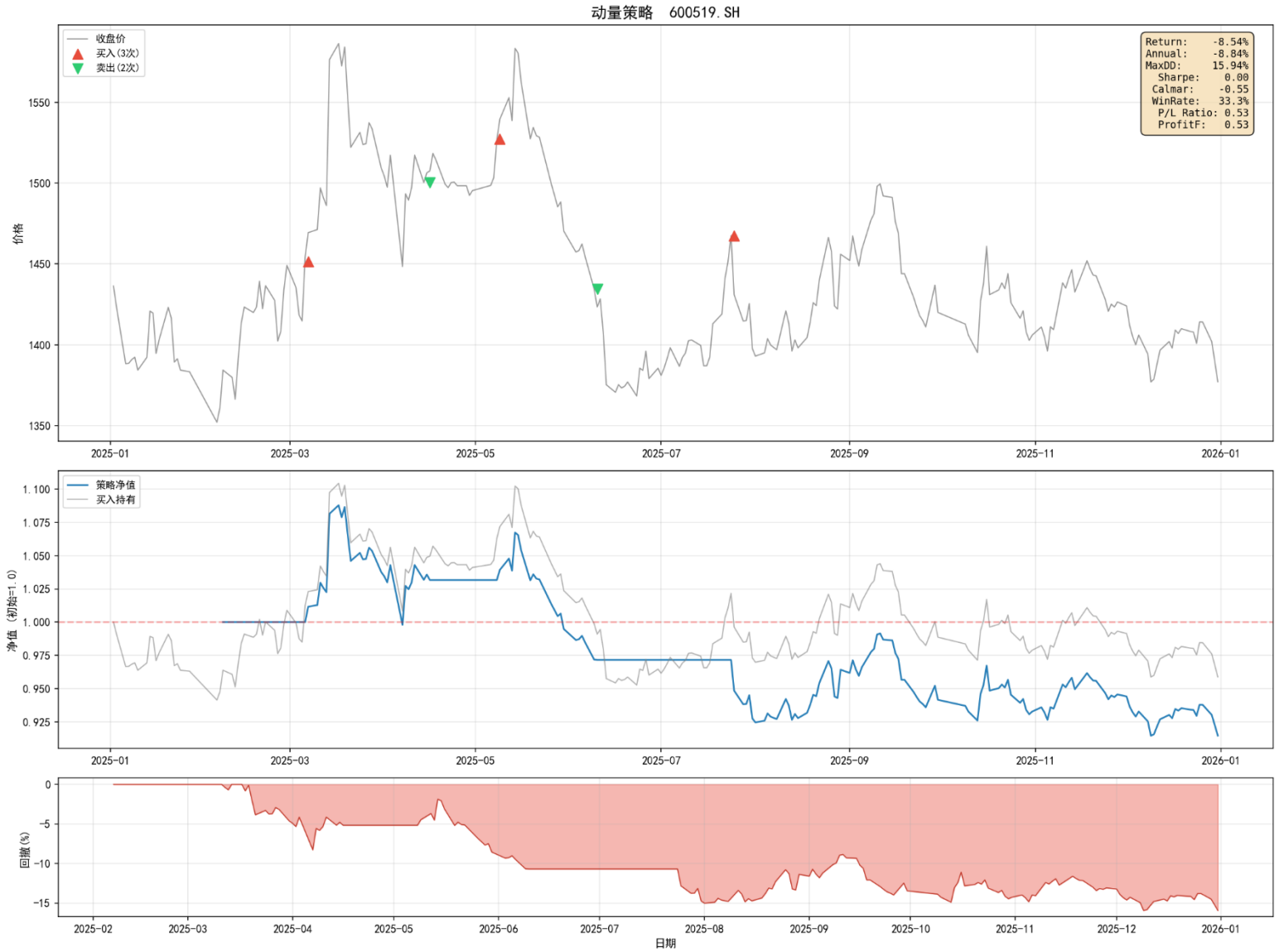
6-动量策略.py

CASE：动量策略（追涨杀跌的强者恒强）

动量策略 | 600519.SH | 2025-01-02 ~ 2025-12-31 | 243个交易日

总收益: -8.54% | 年化: -8.84% | 最大回撤: 15.94% | 夏普: 0.00 | 卡玛: -0.55

交易: 3次 | 胜率: 33.3% | 盈亏比: 0.53 | 利润因子: 0.53 | 最大连亏: 1次



线上系统的动态策略插件机制

如果你是一个量化研究员，每天只跑一两个策略，这当然没问题。

但在真实的线上生产环境（比如我们构建的 Zoe 回测系统）中，我们可能需要同时管理上百个策略因子。

Thinking：每次新增策略，都要去修改主程序的代码并重新发版吗？

TO DO：设计一套动态策略插件机制。

插件化机制的核心理念是解耦。

只需要把写好的策略 Python 文件，直接拖进 strategies/ 目录中。

系统的主程序（7-自定义策略.py）会利用 Python 底层的 importlib.util 模块，

在启动时自动扫描该目录，动态发现并加载所有策略

=> 不需要改动主程序的代码。

线上系统的动态策略插件机制

自定义策略的三步规范

为了让主系统能够正确识别和运行你的策略，任何放入 strategies/ 目录的脚本，需要遵守：

- 命名规范：

在 strategies/ 目录下创建独立的 .py 文件（以 _ 开头的文件会被系统自动跳过）。

- 注册身份证：

必须在文件中定义一个名为 STRATEGY_META 的字典，告诉系统这个策略叫什么、核心逻辑是什么、有哪些默认参数。

=> 这相当于策略的元数据（Metadata），前端页面可以直接读取并展示它。

- 统一方法：

都定义一个名为 Strategy 的类，且继承自 bt.Strategy。

CASE： MACD 底背离策略开发

TO DO：搭建MACD 底背离策略，并进行回测

以贵州茅台 600519.SH为例，回测时间 2025-01-01 到 2025-12-31

Thinking：什么是背离？

之前的 MACD 策略只看简单的金叉死叉 => 容易被骗线。

老手喜欢抓背离——即价格创了新低（还在跌），但 MACD 指标的柱子却没有创新低。

这说明下跌的动能已经衰竭 => 随时可能发生惊天大逆转。

CASE： MACD 底背离策略开发

底背离的数学定义：

需要满足两个条件：

条件	价格走势	MACD走势	数学表达
前提	创N日新低	-	当前最低价 ≤ N日最低价的101%
背离	继续走低	未创新低（抬高）	当前MACD > N日MACD最低值

背离的核心判断

```
# 条件1：价格创30日新低（容忍1%误差，防止毛刺）
at_price_low = self.data.low[0] <= self.price_lowest[0] * 1.01

# 条件2：MACD未创新低（当前值比前期低点高20%以上）
# 注意：乘以0.8意味着只要不低于前期低点的80%就算"背离"
macd_higher = self.macd.macd[0] > self.macd_lowest[0] * 0.8
```

这里用的是 0.8（80%）作为缓冲，属于宽松型背离。

严格背离应该是：

```
# 当前MACD严格高于前低
macd_higher = self.macd.macd[0] > self.macd_lowest[-1]
```

前两个条件是左侧预判（猜底），再加上金叉可以进行右侧确认（抄底）。

CASE： MACD 底背离策略开发

```
import backtrader as bt
# 规范 2：定义策略身份证 (STRATEGY_META)
STRATEGY_META = {
    'name': 'MACD底背离',
    'category': 'custom',
    'desc': '价格创新低但MACD未创新低时买入，捕捉趋势反转',
    'params': {'lookback': 30, 'fast': 12, 'slow': 26, 'signal': 9},
    'logic': '价格创N日新低且MACD未创新低-> 买入; MACD死叉 -> 卖出',
}
# 规范 3：定义 Strategy 类
class Strategy(bt.Strategy):
    params = (('lookback', 30), ('fast', 12), ('slow', 26), ('signal', 9))
    def __init__(self):
        # 预计算 MACD
        self.macd = bt.indicators.MACD(self.data.close, ...)
       # 预计算 N日内的价格最低点 和 MACD最低点
        self.price_lowest = bt.indicators.Lowest(self.data.low, period=self.p.lookback)
        self.macd_lowest = bt.indicators.Lowest(self.macd.macd, period=self.p.lookback)
```

```
    def next(self):
        if not self.position:
            # 核心逻辑：判断底背离
            # 1. 价格跌到了 N 日最低点附近 (容忍 1% 误差)
            at_price_low = self.data.low[0] <= self.price_lowest[0] * 1.01
            # 2. 但是 MACD 的值，明显高于 N 日最低时的 MACD 值 (未创新低)
            macd_higher = self.macd.macd[0] > self.macd_lowest[0] * 0.8
            # 3. 并且 MACD 已经开始拐头向上形成金叉
            golden_cross = self.macd.macd[0] > self.macd.signal[0]

            if at_price_low and macd_higher and golden_cross:
                self.buy() # 动能衰竭，抄底买入！
        else:
            # 死叉平仓
            if (self.macd.macd[0] < self.macd.signal[0] and self.macd.macd[-1] >=
self.macd.signal[-1]):
                self.close()
```

macd_divergence.py

CASE： MACD 底背离策略开发

Thinking：为什么要加1%容差?

- 防止毛刺

市场很少正好砸到前低就反弹，通常会在前低附近（上下1-2%）震荡筑底。

- 参数调整建议：

容忍度	代码写法	适用场景
严格	* 1.00	必须创严格新低，适合震荡市
1%容忍	* 1.01	适合蓝筹股（茅台），波动小
2%容忍	* 1.02	适合科技股，波动大
3%容忍	* 1.03	适合加密货币，波动极大

前低1290是心理支撑位

1298距离支撑仅0.5%，风险收益比依然极好（止损可以放在1280）

如果非要等到1290，可能永远等不到，直接反弹上去了

CASE： MACD 底背离策略开发

MACD底背离 | 600519.SH | 2024-01-02 ~ 2025-12-31 | 481个交易日

总收益: -1.12% | 年化: -0.59% | 最大回撤: 1.12% | 夏普: -4.56 | 卡玛: -0.53

交易: 1次 | 胜率: 0.0% | 盈亏比: 0.00 | 利润因子: 0.00 | 最大连亏: 1次

Summary

当我们把 macd_divergence.py 丢进文件夹，运行主程序 7-自定义策略.py 时

控制台会自动打印出：加载: macd_divergence (MACD底背离)，并立刻启动回测引擎计算它的绩效报告。

到了这一步，我们构建了一个属于自己的量化策略中台。

- Charles 负责底层数据的清洗与入库。
- Zoe 的 Cerebro 引擎负责调度和绩效清算。
- 作为策略研究员的你，未来的工作就是不断地在 strategies/ 目录下新建 .py 文件

=> 把你的灵感翻译成 STRATEGY_META 和 next() 函数。

打卡：搭建你的股票交易回测

你的自选股票，在2025年采用哪种交易策略是适合的

- 可以使用自选股，也可以使用这里的 贵州茅台
- 双均线、MACD策略、RSI策略、布林带策略、动量策略、MACD 底背离策略
- 你为什么觉得哪种策略，最能体现当年的交易行情

股票: 600519.SH

数据: 2025-01-02 ~ 2025-12-31 共243个交易日

==========

初始资金: 1,000,000.00

手续费: 2.0 (万分之)

仓位比例: 95%

最终资金: 910,551.59

总收益率: -8.94%

最大回撤: 10.36%

==========

Thank You

Using data to solve problems